

Основы современного JavaScript для React

Фундаментальный разбор синтаксиса ES6+, модели памяти, функционального программирования, замыканий и принципов иммутабельности, лежащих в основе декларативного UI.

Модель памяти, переменные и области видимости

1. Примитивы против ссылочных типов (Stack vs Heap)

Понимание физики работы памяти в JavaScript — фундамент для понимания механизма рендеров в React. Память браузерного движка V8 разделена на две ключевые области:

- **Стек (Stack Memory):** Статическая память фиксированного размера. В ней хранятся примитивные типы данных (`number`, `string`, `boolean`, `null`, `undefined`, `symbol`, `bigint`), а также указатели (адреса) на объекты. Переменные передаются *по значению* (*by value*).
- **Куча (Heap Memory):** Динамическая память произвольного размера. В ней хранятся сложные структуры: объекты, массивы и функции. Переменные содержат лишь ссылку (адрес) на ячейку в куче. Передача происходит *по ссылке* (*by reference*).

🧠 Физическое распределение памяти V8 Engine:

СТЕК (STACK)

БЫСТРЫЙ / ПРИМИТИВЫ

age	25
isOnline	true
userA (Pointer)	0x004F2A
userB = userA	0x004F2A



КУЧА (HEAP)

ДИНАМИЧЕСКИЕ ОБЪЕКТЫ

Адрес [0x004F2A]:

```
{ id: 1, name: "Иван", role: "Dev" }
```

📌 userA и userB указывают на один физический объект!

🔗 Ссылочная идентичность в React (Object.is)

React сравнивает старое и новое состояние компонента алгоритмом `Object.is` (поверхностное сравнение ссылок). Если вы мутируете объект напрямую: `user.name = "Петр"`, адрес ссылки в стеке не изменится (`0x004F2A === 0x004F2A`), и React **не запустит перерисовку интерфейса**.

2. Области видимости: var vs let vs const

В современном React разработка ведется исключительно с использованием `const` (по умолчанию) и `let` (только если значение переменной действительно будет перезаписано в текущем блоке). Ключевое слово `var` считается устаревшим антипаттерном.

Критерий	<code>var</code> (Устаревший)	<code>let</code> (Современный)	<code>const</code> (Стандарт React)
Область видимости	Функциональная (Function Scope)	Блочная (Block Scope <code>{...}</code>)	Блочная (Block Scope <code>{...}</code>)
Всплытие (Hoisting)	Всплывает со значением <code>undefined</code>	Всплывает в TDZ (Temporal Dead Zone)	Всплывает в TDZ (Temporal Dead Zone)
Переприсвоение	Разрешено	Разрешено	Запрещено (ссылка неизменна)
Мутация свойств	Разрешена	Разрешена	Разрешена (внутри объекта/массива)

Стрелочные функции, контекст `this` и замыкания

→ 1. Стрелочные функции (Arrow Functions)

Стрелочные функции предоставляют компактный синтаксис и обладают принципиальным поведением: они **не создают собственного контекста** `this`, объекта `arguments` и не могут использоваться как конструкторы с `new`.

```
// 1. Полная форма записи
const multiply = (a, b) => {
  return a * b;
};

// 2. Краткая форма с неявным возвратом (Implicit Return)
const double = x => x * 2;

// 3. Неявный возврат объекта (Обязательно оборачивается в круглые скобки!)
// Крайне часто применяется при обновлении стейта в React:
const createUser = (id, name) => ({ id, name, createdAt: Date.now() });
```

2. Замыкания (Closures) и их роль в хуках React

Замыкание — это способность функции запоминать и сохранять доступ к переменным из своей лексической области видимости (Lexical Scope) даже после того, как внешняя функция завершила свое выполнение.

```
function createCounter() {  
  let count = 0; // Переменная сохраняется в куче благодаря замыканию  
  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const counter1 = createCounter();  
console.log(counter1()); // 1  
console.log(counter1()); // 2
```

Как хук `useState` использует замыкания?

Каждый вызов функционального компонента в React создает новую область видимости. Хуки внутри React хранят массив состояний в Fiber-узле. Благодаря механизму замыканий компонент при каждом рендере связывается со своей актуальной ячейкой состояния.

✦ 3. Чистые функции (Pure Functions)

React ожидает, что функциональные компоненты будут чистыми функциями. Функция называется чистой, если она удовлетворяет двум строгим математическим правилам:

1. **Детерминированность:** При одинаковых входных аргументах (`props`) функция всегда возвращает идентичный результат (JSX).
2. **Отсутствие побочных эффектов (Side Effects):** Функция не мутирует переменные вне своей области видимости, не делает сетевых запросов и не меняет DOM во время фазы рендера.

Деструктуризация, Rest и Spread операторы



1. Деструктуризация объектов и массивов

Деструктуризация позволяет извлекать свойства объектов и элементы массивов в отдельные переменные с помощью лаконичного синтаксиса.

Деструктуризация объектов (Props)

Позволяет принимать пропсы сразу именованными переменными с дефолтными значениями:

```
// В React компонентах:  
function UserCard({ name, role = "Guest", age }) {  
  return <div>{name} ({role})</div>;  
}
```

Деструктуризация массивов (useState)

Позиционное извлечение элементов (имена переменных выбирает разработчик):

```
// Хук возвращает кортеж [значение, функция]  
const [count, setCount] = useState(0);  
const [user, setUser] = useState(null);
```

2. Spread оператор (...) и принцип Иммутабельности

Оператор расширения (Spread) создает неглубокую копию (Shallow Copy) объекта или массива, раскладывая его элементы в новую структуру данных.

✗ МУТАЦИЯ (Запрещено в React)

Прямое изменение объекта в памяти. Ссылка в стеке **не меняется**, React считает, что данные те же самые, и **не обновляет экран**.

```
// Мутация стейта
state.push("Новый элемент");
user.age = 26;

// Ссылка осталась: 0x001 ≡ 0x001
// ✗ Рендер НЕ произойдет!
```

✓ ИММУТАБЕЛЬНОСТЬ (Требование React)

Создание **нового объекта/массива** через Spread. Появляется новый адрес в памяти, React фиксирует разницу и **мгновенно ререндерит UI**.

```
// Иммутабельное обновление
const newList = [...state, "Новый"];
const newUser = { ...user, age: 26 };

// Новая ссылка: 0x002 ≠ 0x001
// ✓ React запускает рендер!
```

... 3. Rest параметры (`...rest`) в функциях и объектах

Синтаксис **Rest** выглядит так же, как Spread (три точки `...`), но выполняет строго противоположную задачу: он **собирает оставшиеся свойства или аргументы в единый объект или массив**.

СЦЕНАРИЙ 01



В аргументах функции

Собирает все переданные дополнительные аргументы в полноценный массив:

```
function fn(a, b, ...restArgs)
```

В React: Обработчики событий и утилиты с переменным числом аргументов.

СЦЕНАРИЙ 02



В деструктуризации объекта

Извлекает указанные поля, а все остальные ключи собирает в новый объект:

```
const { id, title, ...otherProps }
```

В React: Проброс неявных HTML-атрибутов (`onClick` , `aria-*` , `disabled`) в целевой тег.

СЦЕНАРИЙ 03



В деструктуризации массива

Забирает первый элемент в переменную, а остаток массива сохраняет в массив:

```
const [first, ...remaining] = ite
```

В React: Разделение списков (главный баннер + список остальных постов).

Практический пример создания гибкого компонента кнопки с пробросом Rest-параметров:

```
// Компонент забирает свои кастомные пропсы (label, variant),  
// а все остальные (onClick, id, disabled, style, aria-*)  
// собирает в объект ...restProps и пробрасывает в нативный HTML-тег <button>:  
function CustomButton({ label, variant = "primary", ...restProps }) {  
  return (  
    <button className={`btn btn-${variant}`} {...restProps}>  
      {label}  
    </button>  
  );  
}
```

```
// Использование в коде:  
<CustomButton  
  label="Купить билет"  
  variant="success"  
  onClick={() => alert("Успешно!")}  
  disabled={false}  
  data-analytics="buy-btn"  
>
```

Иммутабельная трансформация массивов

В React интерфейс строится на базе списков данных. Использование мутирующих методов массивов (`push` , `pop` , `splice` , `shift` , `sort` , `reverse`) запрещено при работе со стейтом. Вместо них используются чистые методы, возвращающие новый массив.

Метод	Что делает	Применение в React
<code>.map()</code>	Трансформирует каждый элемент массива в новый вид.	Преобразование массива данных в массив JSX-элементов <code><li key={...}></code> .
<code>.filter()</code>	Возвращает новый массив только с элементами, прошедшими условие.	Удаление элемента из стейта по ID или реализация живого поиска.
<code>.reduce()</code>	Сворачивает массив в единое значение (число, объект, словарь).	Подсчет общей суммы корзины товаров или группировка данных.
<code>.find()</code> / <code>.some()</code>	Поиск первого совпадения / Проверка наличия хотя бы одного элемента.	Проверка, добавлен ли товар в избранное (возврат <code>boolean</code>).

</> Примеры паттернов обновления списков в React

```
const todos = [  
  { id: 1, text: "Выучить JS", completed: true },  
  { id: 2, text: "Понять React", completed: false }  
];  
  
// 1. Отрисовка списка через .map()  
const todoElements = todos.map(todo => (  
  <li key={todo.id}>{todo.text}</li>  
));  
  
// 2. Удаление элемента по ID через .filter()  
const deleteId = 1;  
const filteredTodos = todos.filter(todo => todo.id !== deleteId);  
  
// 3. Обновление одного элемента в массиве через .map()  
const toggleId = 2;  
const updatedTodos = todos.map(todo =>  
  todo.id === toggleId ? { ...todo, completed: !todo.completed } : todo  
);
```

Современные операторы и условный рендеринг

? 1. Тернарный оператор и логическое И (`&&`)

Внутри JSX нельзя писать классические конструкции `if/else` или циклы `for`, так как JSX принимает только выражения (Expressions), возвращающие значение.

Тернарный оператор (Условие ? A : B)

Выбор между двумя взаимоисключающими блоками интерфейса:

```
{isLoggedIn ? (  
  <UserProfile user={user} />  
) : (  
  <LoginForm />  
)}
```

Логическое И (Условие && Элемент)

Отрисовка блока только при истинности условия:

```
{hasUnreadMessages && (  
  <span className="badge">Новые сообщения!</span>  
)}
```

⚡ Подводный камень: Ловушка нуля (0 && <Component />)

Если левая часть выражения `&&` вычисляется в число `0`, JavaScript возвращает само значение `0`, и на экране вместо скрытия отобразится цифра 0. **Решение:** приводить условие к строгому boolean: `Boolean(count) && <Component />` или `count > 0 && <Component />`.

🐱 2. Опциональная цепочка (`?.`) и нулевое слияние (`??`)

- **Optional Chaining** (`?.`): Безопасное чтение глубоко вложенных свойств объекта без падения с ошибкой «*Cannot read properties of undefined*», если промежуточное свойство равно `null` или `undefined`.
- **Nullish Coalescing** (`??`): Оператор нулевого слияния возвращает правый операнд только в том случае, если левый равен строго `null` или `undefined` (в отличие от `||`, который срабатывает также на `0` и пустую строку `""`).

```
// Безопасное чтение данных с бэкенда:  
const userCity = response?.data?.user?.address?.city ?? "Город не указан";
```

ES-модули и асинхронность (Async / Await)

🔗 1. Модульная система ECMAScript (ESM)

React-проекты строятся на базе нативного синтаксиса модулей стандартов ES6+:

Именованный экспорт / импорт

Экспорт нескольких сущностей из одного файла. При импорте используются точные имена в фигурных скобках:

```
// helpers.js
export const API_URL = "https://api.com";
export function formatPrice(p) { ... }

// App.jsx
import { API_URL, formatPrice } from './helpers';
```

Экспорт по умолчанию (Default)

Главная сущность файла (компонент). Файл может содержать только один `export default`:

```
// Button.jsx
export default function Button() { ... }

// App.jsx (Имя при импорте может быть любым)
import CustomButton from './Button';
```

🔗 2. Асинхронные запросы: Promises и синтаксис `async/await`

Вся работа с сетью (получение JSON с REST API) асинхронна. Синтаксис `async/await` представляет собой синтаксический сахар над промисами, делая асинхронный код визуально линейным и простым для чтения.

```
// Асинхронная загрузка данных для использования внутри useEffect
async function fetchProducts() {
  try {
    const response = await fetch('https://api.example.com/products');

    // Проверяем HTTP статус (200-299)
    if (!response.ok) {
      throw new Error(`Ошибка HTTP: ${response.status}`);
    }

    const data = await response.json();
    return data;
  } catch (error) {
    console.error("Сетевой сбой при загрузке:", error.message);
    throw error;
  }
}
```